Tarea 11: Tipos de datos en PostgreSQL

Molina Ramírez César Alejandro

Tipos numéricos

- **smallint** (int2) ocupa 2 bytes y almacena enteros en el intervalo de −32 768 a 32 767. Por ejemplo, `SELECT 42::smallint;` devuelve 42. Es adecuado para valores de rango reducido, como la edad de una persona.
- **integer** (int, int4) ocupa 4 bytes y almacena enteros en el intervalo de −2 147 483 648 a 2 147 483 647. Es el tipo habitual para cantidades, como las piezas disponibles en un inventario. Los números almacenados como `varchar` no se ordenan numéricamente: "10" precede a "9".
- **bigint** (int8) ocupa 8 bytes y almacena enteros en el intervalo de −9 223 372 036 854 775 808 a 9 223 372 036 854 775 807. Se utiliza cuando el rango de `integer` resulta insuficiente, por ejemplo en identificadores de tablas muy grandes.
- **numeric(p,s)** (decimal) almacena números exactos con una precisión de hasta 1000 dígitos, de los cuales `s` corresponden a la parte decimal. Sin precisión declarada, admite hasta 131 072 dígitos antes del punto decimal y 16 383 después. Por ejemplo, `numeric(6,2)` almacena 1234.5 como 1234.50. Es el tipo indicado para cantidades monetarias, como saldos, precios y facturas.
- **double precision** (float8) es un número de punto flotante de 8 bytes con una precisión de al menos 15 dígitos decimales. Al ser aproximado, no es adecuado para cantidades monetarias: por ejemplo, `0.1 + 0.2` produce 0.30000000000000004.

Tipos de carácter

- **character(n)** (char(n)) almacena cadenas de longitud fija, rellenadas con espacios hasta `n` caracteres. Admite hasta 10 485 760 caracteres. Por ejemplo, "ab" en un `char(5)` se almacena como "ab" seguido de tres espacios.
- **character varying(n)** (varchar(n)) almacena cadenas de longitud variable con un límite de `n` caracteres, que no puede superar 10 485 760. Si en una columna `varchar(3)` se intenta almacenar "Ana López", PostgreSQL devuelve un error.
- **text** almacena cadenas de longitud variable sin límite declarado, de hasta 1 GB. Salvo el espacio adicional que ocupa `char` y el costo de verificar la longitud, las tres variantes no presentan diferencias de rendimiento.

Tipos de fecha y hora

- **date** almacena una fecha de calendario en formato AAAA-MM-DD, en el intervalo del 4713 a. C. al 5 874 897 d. C., con 4 bytes. Por ejemplo, `SELECT DATE '2026-11-06';` devuelve `2026-11-06`. A diferencia de una cadena de texto, PostgreSQL valida que la fecha exista, por ejemplo, rechaza el 30 de febrero, y permite ordenar fechas y calcular diferencias entre ellas.
- **timestamp** almacena fecha y hora con 8 bytes y una resolución de 1 microsegundo, en el intervalo del 4713 a. C. al 294 276 d. C. Cuando la zona horaria es relevante, PostgreSQL ofrece `timestamp with time zone` (timestamptz), que almacena el valor en UTC.
- **interval** almacena duraciones con 16 bytes, en un rango de ±178 000 000 años. Por ejemplo, `TIMESTAMP '2026-11-06' - TIMESTAMP '2026-10-01'` devuelve `36 days`.

Otros tipos

- **bytea** almacena cadenas binarias de longitud variable, de hasta 1 GB. Se escribe en formato hexadecimal con el prefijo `\x`, por ejemplo `'\xDEADBEEF'`, o en formato escape. Para datos de gran tamaño, PostgreSQL también ofrece los objetos grandes (large objects).
- **serial** (smallserial, bigserial) no es un tipo de dato propiamente dicho, sino una notación abreviada que crea una columna de tipo integer (smallint o bigint) con una secuencia que asigna valores automáticamente, de 1 a 2 147 483 647 en el caso de serial. Por ejemplo, en una tabla de alumnos, la clave primaria puede declararse como `id SERIAL PRIMARY KEY`. Las columnas de identidad (`GENERATED ALWAYS AS IDENTITY`) son la alternativa conforme al estándar SQL.
- **enum** representa un conjunto ordenado de valores de texto definido por el usuario mediante `CREATE TYPE`. Ocupa 4 bytes por valor, y cada etiqueta puede tener hasta 63 bytes. Por ejemplo, `CREATE TYPE estado_pedido AS ENUM ('pendiente', 'enviado', 'entregado');` define tres valores. Los valores siguen el orden de su declaración y pueden agregarse con `ALTER TYPE ... ADD VALUE`.

Referencias

- PostgreSQL Global Development Group, *PostgreSQL Documentation: Data Types*, capítulo 8.